

Appendix A

Implementation of the EIT cooling simulations

The numerical simulations were implemented in Python using the QuTiP framework. Two models were considered: an idealized three-level Λ system and a complete multilevel model of the ^{87}Rb D_2 transition. The following code excerpts summarize the main steps of the implementation. Plotting routines are not included.

A.1 Three-level Λ system

The physical and numerical parameters of the idealized model are defined in `config.py`. Frequencies are expressed in units of the spontaneous decay rate γ .

Listing A.1: *Main parameters of the three-level simulation.*

```
gamma = 1.0
nu = 0.5 * gamma
Delta_c = 15.0 * gamma
Delta_p = Delta_c
Omega_c = np.sqrt(4 * np.abs(Delta_c) * nu)
Omega_p = 0.1 * gamma
eta = 0.35
N_vib = 25
psi0 = tensor(basis(3, 0), fock(N_vib, 15))
```

The atomic Hilbert space is composed of the states $|g_1\rangle$, $|g_2\rangle$ and $|e\rangle$, while the motional degree of freedom is represented by a truncated harmonic oscillator. The Hamiltonian includes the trap energy, the coupling and probe fields, and the spatial dependence of the probe interaction in the Lamb–Dicke regime.

Listing A.2: *Hamiltonian and collapse operators for the three-level model.*

```
a = tensor(qeye(3), destroy(config.N_vib))
n_op = a.dag() * a
sig_eg1 = tensor(basis(3, 2) * basis(3, 0).dag(), qeye(config.N_vib))
sig_eg2 = tensor(basis(3, 2) * basis(3, 1).dag(), qeye(config.N_vib))
```

```

proj_g1 = tensor(basis(3, 0) * basis(3, 0).dag(), qeye(config.N_vib))
proj_g2 = tensor(basis(3, 1) * basis(3, 1).dag(), qeye(config.N_vib))

H = (config.Delta_p * proj_g1 + config.Delta_c * proj_g2 + config.nu * n_op
     + (config.Omega_c / 2.0) * (sig_eg2 + sig_eg2.dag())
     + (config.Omega_p / 2.0) * (sig_eg1 * (1 + 1j * config.eta * (a + a.dag()))
                               + sig_eg1.dag() * (1 - 1j * config.eta * (a + a.dag()))))

c_ops = [np.sqrt(config.gamma / 2) * sig_eg1.dag(),
         np.sqrt(config.gamma / 2) * sig_eg2.dag()]

result = mesolve(H, config.psi0, config.t_list, c_ops, [n_op])

```

The steady-state Fano profile is obtained by scanning the probe detuning while keeping the coupling field fixed. At each detuning, the steady-state density matrix is calculated and the excited-state population is used as a measure of absorption.

Listing A.3: *Calculation of the steady-state Fano profile.*

```

det_scan = np.linspace(config.Delta_c - 1.5, config.Delta_c + 1.5, 400)
spec = []

for d in det_scan:
    H_at = (+d * basis(3, 0) * basis(3, 0).dag()
           + config.Delta_c * basis(3, 1) * basis(3, 1).dag()
           + (config.Omega_c / 2) * (basis(3, 2) * basis(3, 1).dag()
                                   + basis(3, 1) * basis(3, 2).dag())
           + (config.Omega_p / 2.0) * (basis(3, 2) * basis(3, 0).dag()
                                   + basis(3, 0) * basis(3, 2).dag()))
    c_ops_at = [np.sqrt(config.gamma / 2) * basis(3, 0) * basis(3, 2).dag(),
               np.sqrt(config.gamma / 2) * basis(3, 1) * basis(3, 2).dag()]
    rho_s = steadystate(H_at, c_ops_at)
    spec.append(expect(basis(3, 2) * basis(3, 2).dag(), rho_s))

```

A.2 Full multilevel ^{87}Rb model

The multilevel simulation includes the hyperfine and Zeeman structure of the ^{87}Rb D_2 line. The axial configuration is defined by the trap frequency, the Lamb–Dicke parameter, the magnetic field, and the Rabi frequencies of the probe, coupling and repump lasers.

Listing A.4: *Main parameters of the multilevel axial simulation.*

```

gamma = 1.0
MHz = 1 / 6.067
nu = 0.450 / 6.067 * gamma
eta = 0.09
n_phon = 2
N_vib = 10

B_field = 4.0
mu_B = 1.399 * MHz
g_g1, g_g2 = -0.5, 0.5
m_g1, m_g2 = -1, -1

```

```

Delta_c = 13 * gamma
zeeman_offset = (g_g2 * m_g2 - g_g1 * m_g1) * mu_B * B_field
Delta_p_center = Delta_c - zeeman_offset

Omega_c_amp = np.sqrt(4 * nu * (nu + np.abs(Delta_c)))
Omega_p_amp = 0.09 * gamma
Omega_r_amp = 0.3 * gamma

pol_p = -1 # sigma-
pol_c = -1 # sigma-
pol_r = 0 # pi

```

The coupling strengths of the allowed transitions are calculated from the hyperfine dipole matrix elements. Each laser Rabi frequency is normalized to its target transition, while the other transitions are weighted according to their relative dipole amplitudes.

Listing A.5: *Relative laser coupling in the multilevel model.*

```

_, Fg_c, mg_c = cfg.COUPLING_GROUND
_, Fe_t, me_t = cfg.EXCITED_TARGET
d_c_target = cfg.dipole_element(Fg_c, mg_c, Fe_t, me_t, cfg.pol_c)

for gi in g1_idxes:
    Fg_i, mg_i = cfg.atom_labels[gi][1], cfg.atom_labels[gi][2]
    for ei in e_idxes:
        Fe_i, me_i = cfg.atom_labels[ei][1], cfg.atom_labels[ei][2]
        d = cfg.dipole_element(Fg_i, mg_i, Fe_i, me_i, cfg.pol_c)
        if abs(d) > 1e-8:
            c_rel = d / d_c_target
            V = (cfg.Omega_c_amp * c_rel / 2.0) * basis(cfg.N_atom, ei) * basis(cfg.N_atom, gi).dag()
            H_const += V + V.dag()

```

Spontaneous emission is described by collapse operators connecting each excited state to all dipole-allowed ground states. The branching fractions are normalized for every excited sublevel.

Listing A.6: *Spontaneous-emission operators in the multilevel model.*

```

c_ops = []

for ei in e_idxes:
    Fe_i, me_i = cfg.atom_labels[ei][1], cfg.atom_labels[ei][2]
    for gi in g1_idxes + g2_idxes:
        Fg_i, mg_i = cfg.atom_labels[gi][1], cfg.atom_labels[gi][2]
        b = cfg.branching_fraction(Fg_i, mg_i, Fe_i, me_i)
        if b > 1e-10:
            c_ops.append(np.sqrt(cfg.gamma * b) * (basis(cfg.N_atom, gi) * basis(cfg.N_atom, ei).dag()))

```

The Fano spectrum is calculated by constructing the Hamiltonian for each value of the probe detuning and solving the corresponding steady-state master equation. The total excited-state population is used as the spectroscopic signal.

Listing A.7: *Steady-state Fano calculation for the multilevel model.*

```

d_p_target = cfg.dipole_element(Fg_p, mg_p, Fe_t, me_t, cfg.pol_p)

for dp in det_list:

```

```

H_probe = 0
two_photon_det = dp - cfg.Delta_c

for gi in g2_idxxs:
    H_probe += two_photon_det * (basis(cfg.N_atom, gi) * basis(cfg.N_atom, gi).dag())
    Fg_i, mg_i = cfg.atom_labels[gi][1], cfg.atom_labels[gi][2]
    for ei in e_idxxs:
        Fe_i, me_i = cfg.atom_labels[ei][1], cfg.atom_labels[ei][2]
        d = cfg.dipole_element(Fg_i, mg_i, Fe_i, me_i, cfg.pol_p)
        if abs(d) > 1e-8:
            c_rel = d / d_p_target
            V = (cfg.Omega_p_amp * c_rel / 2.0) * basis(cfg.N_atom, ei) * basis(cfg.N_atom, gi).dag()
            H_probe += V + V.dag()

rho_ss = steadystate(H_const + H_probe, c_ops)
results["abs_tot"].append(expect(P_e_total, rho_ss))

```

For the time evolution, the atomic and motional Hilbert spaces are combined through a tensor product. The probe interaction contains the motional operator associated with the Lamb–Dicke expansion, whereas the coupling and repump fields are treated as transverse fields.

Listing A.8: *Hamiltonian construction for the Monte Carlo simulation.*

```

a = tensor(qeye(cfg.N_atom), destroy(N_vib))
n_op = a.dag() * a

two_photon_det = dp - cfg.Delta_c
LD_term_plus = qeye(N_vib) + 1j * eta * (destroy(N_vib) + destroy(N_vib).dag())
LD_term_minus = qeye(N_vib) - 1j * eta * (destroy(N_vib) + destroy(N_vib).dag())

H_probe += tensor(two_photon_det * sum([basis(cfg.N_atom, gi) * basis(cfg.N_atom, gi).dag() for gi in
    g2_idxxs]), qeye(N_vib))

for gi in g2_idxxs:
    Fg_i, mg_i = cfg.atom_labels[gi][1], cfg.atom_labels[gi][2]
    for ei in e_idxxs:
        Fe_i, me_i = cfg.atom_labels[ei][1], cfg.atom_labels[ei][2]
        d = cfg.dipole_element(Fg_i, mg_i, Fe_i, me_i, cfg.pol_p)
        if abs(d) > 1e-8:
            c_rel = d / d_p_target
            V_atom = (cfg.Omega_p_amp * c_rel / 2.0) * basis(cfg.N_atom, ei) * basis(cfg.N_atom, gi).dag()
            H_probe += tensor(V_atom, LD_term_plus) + tensor(V_atom.dag(), LD_term_minus)

H_trap = cfg.nu * n_op
H_tot = H_const + H_probe + H_trap + H_repump

```

The initial state is chosen with the atom in the probe ground state and the motional mode in a Fock state containing the specified initial number of phonons. The dynamics are then obtained by averaging several Monte Carlo quantum trajectories.

Listing A.9: *Initial state and Monte Carlo evolution.*

```

ideal_g_probe = cfg.atom_labels.index(cfg.PROBE_GROUND)
psi0 = tensor(basis(cfg.N_atom, ideal_g_probe), fock(N_vib, n_phon))

t_list_full = np.linspace(0.0, t_total, n_points)

```

```

state_projectors = [tensor(basis(cfg.N_atom, i) * basis(cfg.N_atom, i).dag(), qeye(N_vib)) for i in range(
    cfg.N_atom)]
e_ops = [n_op] + state_projectors

result = make_mcsolve_call(H_tot, psi0, t_list_full, c_ops, e_ops, n_traj)
n_expect_full = result.expect[0]

```

These excerpts show the main numerical differences between the two models. The three-level simulation isolates the EIT cooling mechanism, whereas the multilevel model includes hyperfine transitions, Zeeman shifts, optical pumping, repumping, and the corresponding Monte Carlo dynamics.